ODACE CONSULTING
La synergie des expertises.

ABIDJAN · PARIS · CASABLANCA

GUIDE DE RÉALISATION

Plateforme d'orchestration des tests

Construire la solution pas à pas : JMeter, Selenium, API, orchestrateur, agents

PUBLIC	Développeurs et architecte du projet. Document interne de travail : aucun montant, aucun engagement commercial

Version : **2.1 détaillée et illustrée — 23 octobre 2026** Statut : **à valider avec l'architecte avant l'étape 1**

Comment lire ce guide

Chaque étape donne un objectif, les tâches à réaliser dans l'ordre, les livrables et un critère de fin vérifiable. On ne démarre pas l'étape n+1 tant que le critère de fin de l'étape n n'est pas atteint. Les extraits de code montrent la forme attendue des contrats, pas une implémentation complète. Les choix marqués « à valider » dépendent du cadrage client.

Sommaire

N°	CONTENU
0	Comprendre la solution : problème, fonctionnement, parcours utilisateur
A	Principes de conception, stack et organisation du dépôt
0	Socle : environnement, dépôt, intégration continue
1	Modèle de données et migrations
2	API REST, authentification, rôles
3	Orchestrateur : états, file, verrous, idempotence
4	Agents d'exécution et contrat de connecteur
5	Connecteur JMeter
6	Connecteur Selenium
7	Workflows, planification, webhooks, quality gates
8	Interface web
9	Sécurité, supervision, sauvegarde
10	Tests de la plateforme et recette
B	Règles d'équipe, définition de « terminé », lot CI/CD optionnel

ODACE CONSULTING SARLU — Société à Responsabilité Limitée Unipersonnelle
Capital social : 10 000 000 FCFA · Société créée en 2022 · RCCM CI-ABJ-03-2022-B13-00315
N° Compte Contribuable 2208051 K · Numéro IDU CI-2022-0016887 M · N° employeur CNPS 399695
Siège social : Rue des Banques, 28 BP 628 Abidjan 28, Côte d'Ivoire
Bureau : 24 Rue Alphonse Daudet, face BGFI Bank, Plateau — Abidjan
Côte d'Ivoire : 05 05 13 12 06 · 07 08 96 96 09 — France : +33 6 03 77 08 93
contact@odaceconsulting.com · www.odaceconsulting.com — Abidjan (Côte d'Ivoire) · Paris (France) · Casablanca (Maroc)

0

Comprendre la solution

En une phrase

La plateforme est un point central qui prépare, déclenche, enchaîne et suit les tests logiciels réalisés avec JMeter (charge et performance) et Selenium (parcours fonctionnels), et qui rend un verdict lisible par les humains comme par les pipelines de livraison.

Le problème qu'elle résout

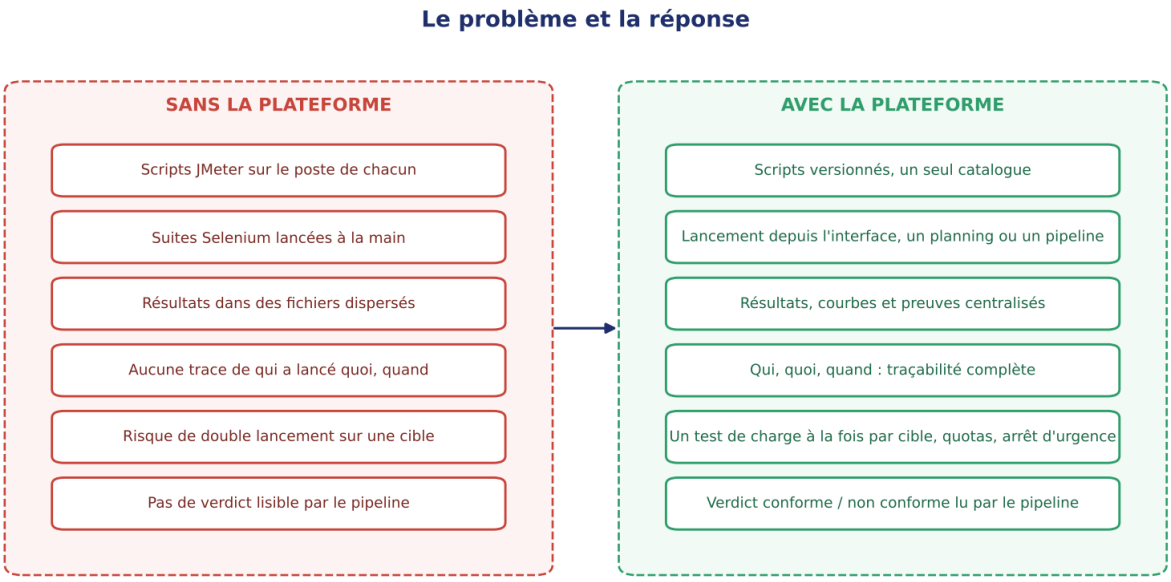


Figure 0.1. Avant et après : ce que change la plateforme.

PROBLÈME RENCONTRÉ	CONSÉQUENCE	RÉPONSE DE LA PLATEFORME
Chaque équipe lance ses tests avec ses propres outils et scripts	Résultats non comparables, savoir-faire perdu au départ d'une personne	Catalogue unique de scripts versionnés, paramètres explicites
Un test de charge lancé deux fois ou au mauvais moment	Application de recette saturée, équipes bloquées	Un test de charge à la fois par cible, plages horaires, quotas, arrêt d'urgence
Générateur de charge saturé sans que personne s'en aperçoive	Chiffres faux qui accusent l'application à tort	Surveillance du générateur ; résultat marqué non fiable
Pas de trace de qui a lancé quoi	Impossible d'expliquer un incident ou de prouver une recette	Journal d'audit et preuves exportables
Les pipelines ne savent pas si les tests de performance sont bons	Mises en production sans verdict objectif	Quality gate : verdict conforme ou non conforme par API et webhook
Selenium utilisé pour mesurer la performance	Mesures sans valeur	Chaque outil garde son rôle ; le connecteur Selenium refuse la montée en charge

Comment elle fonctionne, en clair

1. L'utilisateur choisit une application, un environnement et un script dans l'interface, ou un pipeline fait la même demande par l'API.

2. La plateforme vérifie que la demande est permise : droits de la personne, cible autorisée, plage horaire, quotas, arrêt d'urgence inactif, cible libre.

3. Elle place la demande en file ; un agent d'exécution la prend et la fait tourner dans un conteneur isolé avec le bon outil (JMeter ou Selenium).

4. Pendant le test, elle suit l'état, recueille les journaux et surveille que le générateur de charge ne sature pas.

5. À la fin, elle normalise les résultats, les compare aux seuils définis, et calcule un verdict.

6. Elle conserve tout (script, version, paramètres non secrets, résultats, auteur, horodatage) et met le verdict à disposition de l'humain et du pipeline.

7. Un responsable valide ou refuse le résultat ; cette décision humaine reste distincte du verdict calculé.

Les étapes côté utilisateur

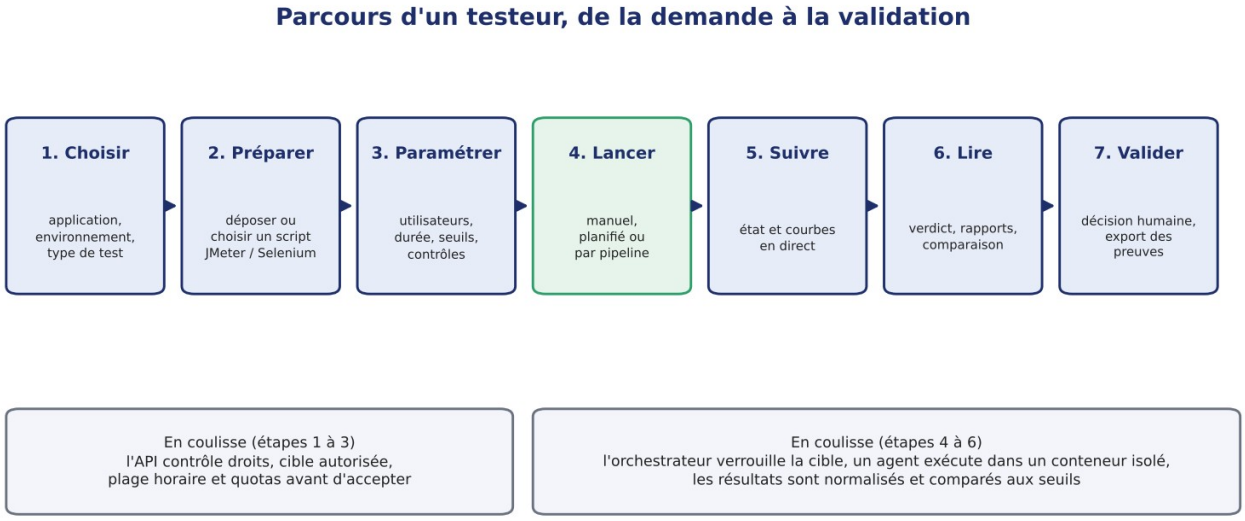


Figure 0.2. Parcours d'un testeur et ce qui se passe en coulisse.

PROFIL	CE QU'IL FAIT AVEC LA PLATEFORME
Testeur	Dépose les scripts, lance les campagnes, suit l'exécution, lit les résultats
Développeur	Déclenche des tests sur les cibles autorisées, consulte les verdicts de ses livraisons
Tech lead	Définit les seuils bloquants, valide les résultats
Responsable	Crée projets et équipes, fixe les quotas
Exploitation	Surveille les agents, dispose de l'arrêt d'urgence
Auditeur	Consulte et exporte les preuves, sans pouvoir modifier
Pipeline CI/CD	Déclenche par webhook signé, lit le verdict, bloque ou autorise la suite

Un exemple concret

Une équipe livre une nouvelle version de l'application de gestion. Le pipeline du client construit l'application, la déploie en recette, puis appelle la plateforme. La plateforme lance d'abord le plan JMeter de charge (200 utilisateurs pendant dix minutes) sur la recette. Si le temps de réponse au 95e centile dépasse le seuil bloquant, elle répond « non conforme » et le pipeline s'arrête. Sinon, elle enchaîne la suite Selenium qui vérifie les parcours métier. Le tech lead consulte le rapport, valide, et la décision est archivée avec les preuves.

Ce que la solution ne fait pas

- Elle ne remplace ni JMeter ni Selenium : elle les pilote.
- Elle ne décide pas de la mise en production : elle fournit un verdict, la décision reste humaine.
- Elle ne relance jamais seule un test de charge après un incident.
- Elle ne déploie pas les applications dans la première version (option lot CI/CD).
- Elle n'écrit pas les scripts de test à la place des équipes.

Les trois verdicts à ne pas confondre

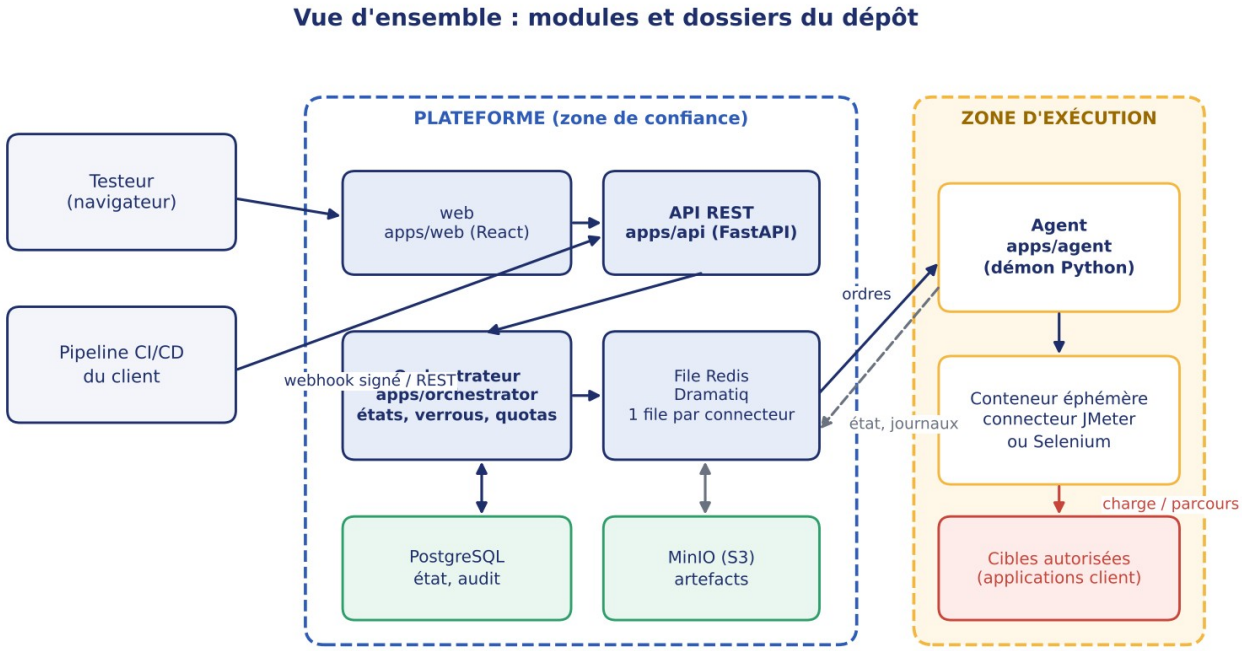
DIMENSION	VALEURS	QUI LA DÉCIDE
État d'exécution	terminée, en erreur, annulée, indéterminée	L'orchestrateur
Résultat	conforme, non conforme, informatif, non fiable	Le calcul de quality gate
Validation	en attente, validée, refusée	Un humain (tech lead)

A

Principes, stack et organisation

Règles de conception à ne jamais contourner

- Une exécution a toujours un état explicite et une seule machine à états, dans l'orchestrateur. Ni l'interface ni les agents ne changent un état directement.
- Le verdict est en trois dimensions : état d'exécution (terminée, en erreur, annulée, indéterminée), résultat (conforme, non conforme, informatif) et validation humaine. On ne les fusionne jamais dans une seule colonne.
- Aucun relancement automatique d'un test de charge après incident : l'état devient « indéterminé » et un humain décide.
- Toute demande de lancement porte une clé d'idempotence ; un même appel répété ne crée qu'une exécution.
- Un seul test de charge à la fois par cible (verrou). Les quotas et plages horaires sont contrôlés avant l'envoi à l'agent, pas pendant.
- Les secrets ne transitent jamais par les journaux, les rapports ou la base : seulement des références vers le coffre.
- JMeter génère de la charge ; Selenium ne génère pas de charge. Le connecteur Selenium refuse tout paramètre de montée en charge.



Les agents n'ouvrent aucun port entrant : ils interrogent l'orchestrateur (flux sortant uniquement).

Figure A. Vue d'ensemble : composants, zones de confiance et dossiers du dépôt.

Stack retenue

COUCHE	CHOIX	REMARQUE
API	Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2, Alembic	OpenAPI généré ; typage strict (mypy)
File et tâches	Redis + Dramatiq (ou Celery, à valider)	Une file par type de connecteur
Base	PostgreSQL 16	Verrous consultatifs pour les cibles
Artefacts	MinIO (S3)	Un préfixe par exécution
Identité	Keycloak, OIDC	Rôles portés par les groupes
Secrets	Vault (ou équivalent du client)	Jetons de courte durée pour les agents
Agents	Python + Docker	Images versionnées : JMeter, Selenium, outils
Interface	React, TypeScript, Vite, TanStack Query	Client API généré depuis OpenAPI
Supervision	Prometheus, Grafana, OpenTelemetry	Un identifiant de corrélation par exécution

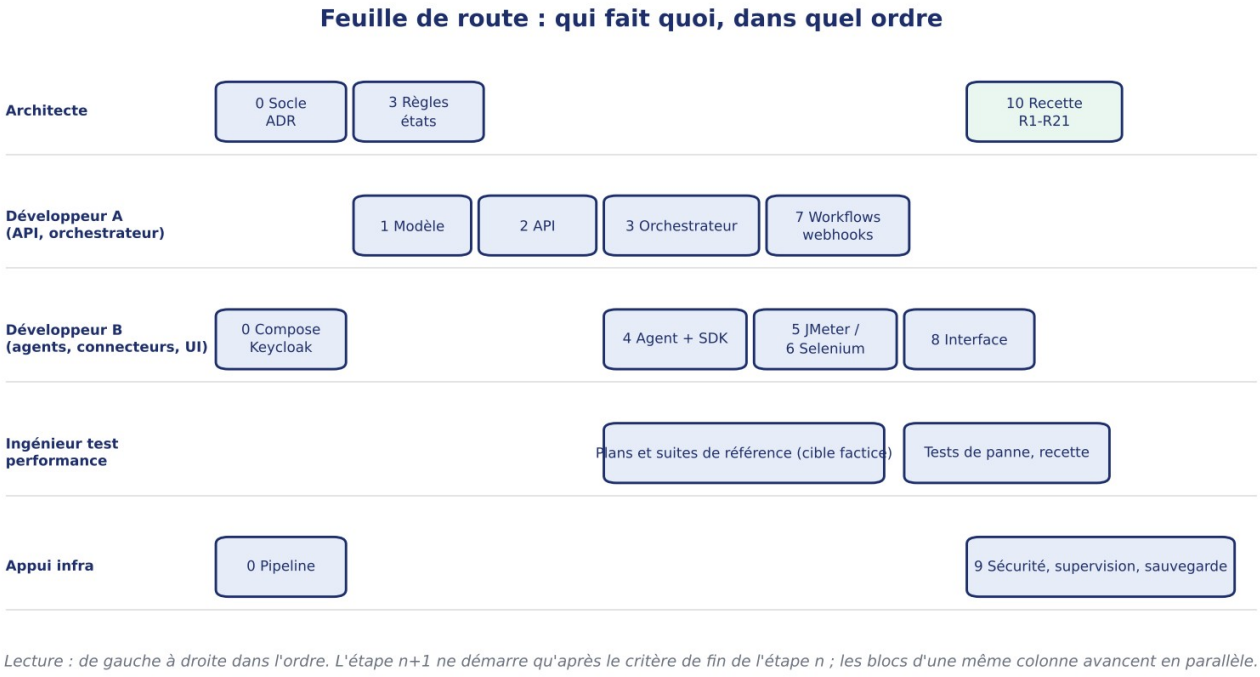


Figure B. Feuille de route : qui fait quoi et dans quel ordre (sans durée ni coût).

Organisation du dépôt (monorepo)

```
orchestra/
├─ apps/
│  ├─ api/            # FastAPI : routes, schémas, auth
│  ├─ orchestrator/   # machine à états, planificateur, verrous
│  ├─ agent/          # démon d'exécution et chargement des connecteurs
│  └─ web/            # interface React
├─ connectors/
│  ├─ sdk/            # interface Connector, modèles, utilitaires
│  ├─ jmeter/
│  └─ selenium/
├─ libs/              # domaine partagé : modèles, événements, erreurs
├─ migrations/        # Alembic
├─ deploy/            # docker-compose, images, manifestes
├─ tests/             # unitaires, intégration, bout en bout
└─ docs/              # ADR, OpenAPI, guides d'exploitation
```

ÉTAPE 0

Socle : environnement, dépôt, intégration continue

Objectif : Qu'un développeur obtienne une plateforme vide mais fonctionnelle en une commande, et que chaque commit soit vérifié automatiquement.

À faire (vue d'ensemble)

- Créer le monorepo, les conventions (ruff, mypy, pytest, eslint, prettier), les hooks de pré-commit et le modèle de revue de code.
- Écrire le docker-compose de développement : PostgreSQL, Redis, MinIO, Keycloak préconfiguré avec les sept rôles, Prometheus, Grafana.
- Ajouter une cible factice (petit service HTTP) pour tester sans toucher aux applications du client.
- Mettre en place le pipeline : lint, types, tests unitaires, construction des images, analyse des dépendances.
- Rédiger les premiers ADR (choix de la file, du modèle de verrous, du format de preuve).

Pourquoi : tout le reste du projet dépend d'un environnement reproductible. Une plateforme de test non reproductible produit des résultats que personne ne peut vérifier.

Pas à pas

1. Créer le dépôt et la structure de dossiers du chapitre A ; ajouter .gitignore, .editorconfig et un fichier CODEOWNERS (l'architecte relit apps/orchestrator et connectors/sdk).
2. Installer les outils : Python 3.12, uv ou poetry, Node 20, Docker, make. Fixer les versions dans .tool-versions.
3. Écrire le Makefile avec les cibles ci-dessous ; chaque développeur ne doit connaître que ces cibles pour travailler.
4. Écrire docker-compose.dev.yml (services du tableau ci-dessous) avec des volumes nommés et des vérifications de santé.
5. Préconfigurer Keycloak par un export de realm : sept groupes (admin, responsable, tech lead, développeur, testeur, exploitation, auditeur) et un utilisateur de test par rôle.
6. Créer la cible factice (apps/fake-target) : un service HTTP avec des routes lentes, en erreur et aléatoires, pour valider les mesures.
7. Construire le pipeline : lint, types, tests, construction des images, analyse des vulnérabilités ; il bloque la fusion s'il échoue.

```
# Makefile (extrait)
up:      ; docker compose -f deploy/docker-compose.dev.yml up -d
down:    ; docker compose -f deploy/docker-compose.dev.yml down
lint:    ; ruff check . && mypy apps libs connectors
test:    ; pytest -q tests/unit tests/integration
migrate: ; alembic upgrade head
seed:    ; python -m libs.seed
```

SERVICE	IMAGE CONSEILLÉE	PORT LOCAL	VÉRIFICATION DE SANTÉ
postgres	postgres:16	5432	pg_isready
redis	redis:7	6379	redis-cli ping
minio	minio/minio	9000 / 9001	/minio/health/live
keycloak	quay.io/keycloak/keycloak	8081	/health/ready

SERVICE	IMAGE CONSEILLÉE	PORT LOCAL	VÉRIFICATION DE SANTÉ
prometheus + grafana	prom/prometheus, grafana/grafana	9090 / 3000	/-/ready
fake-target	construite localement	8088	/health

Pièges fréquents

- Ne pas partager un environnement de développement commun : chacun a le sien, créé par « make up ».
- Ne pas pointer les premiers tests contre une application réelle : la cible factice existe pour cela.
- Figer les versions d'images (pas de « latest »), sinon un test qui passe aujourd'hui échoue demain sans changement de code.

Livrables : dépôt, compose de dev, pipeline vert, README de démarrage.

Critère de fin : un nouveau développeur lance « make up » et voit l'API, Keycloak et Grafana répondre ; le pipeline passe sur la branche principale.

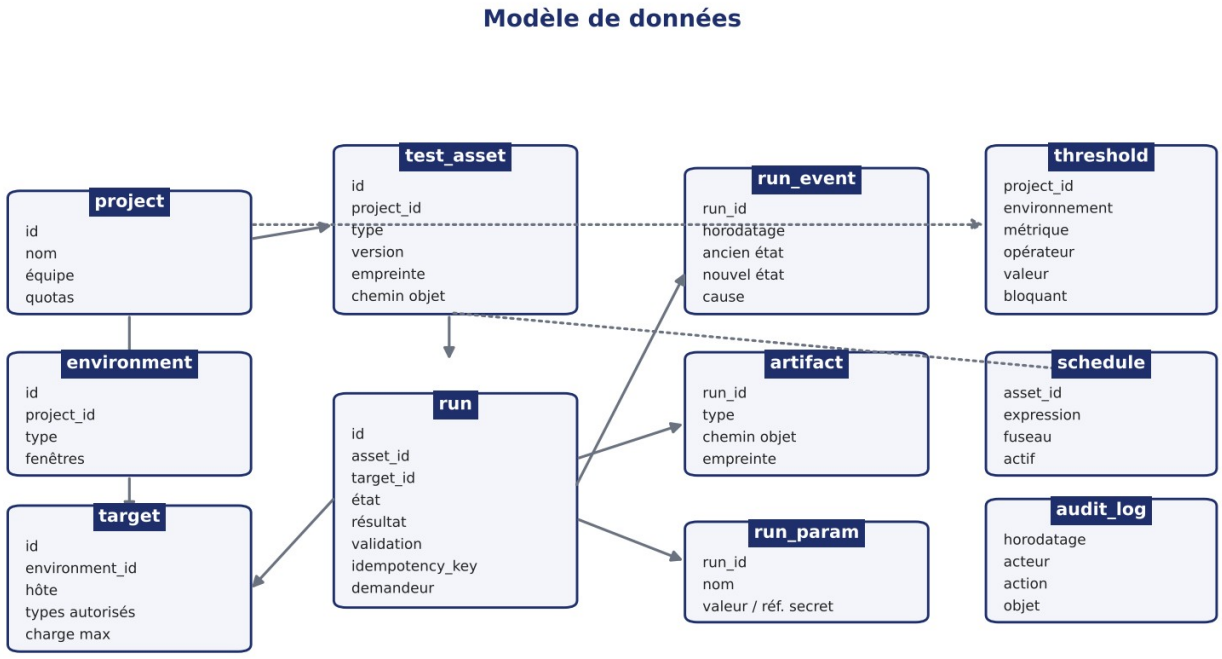
ÉTAPE 1

Modèle de données et migrations

Objectif : Poser les entités qui portent tout le reste, avant d'écrire la moindre route.

À faire (vue d'ensemble)

- Créer les tables avec Alembic, une migration par lot cohérent, jamais de modification d'une migration déjà fusionnée.
- Ajouter les contraintes d'intégrité : clés étrangères, unicité de la clé d'idempotence par projet, énumérations d'états côté base.
- Écrire les modèles SQLAlchemy et un jeu de données de démonstration.
- Ajouter la table d'audit en écriture seule (aucune mise à jour ni suppression applicatives).



Trait plein : clé étrangère. Pointillé : rattachement logique. audit_log est en écriture seule.

Figure 1. Modèle de données : les douze tables et leurs liens.

Pourquoi : l'état d'une exécution, sa traçabilité et l'idempotence sont garantis par la base, pas par le code applicatif seul.

Pas à pas

1. Créer la base et l'utilisateur applicatif (droits minimaux : pas de DROP ni de TRUNCATE).
2. Écrire une première migration par groupe de tables : référentiel (project, environment, target), scripts (test_asset), exécution (run, run_param, run_event, artifact), règles (threshold, schedule, webhook_sub), audit (audit_log).
3. Définir les états en type énuméré PostgreSQL pour empêcher une valeur inconnue.
4. Ajouter l'index unique (project_id, idempotency_key) sur run : c'est lui qui rend l'idempotence fiable même en cas de deux appels simultanés.
5. Retirer à l'utilisateur applicatif les droits UPDATE et DELETE sur audit_log et run_event (écriture seule).
6. Écrire le script de données de démonstration : un projet, deux environnements, une cible factice, un plan JMeter, une suite Selenium.

```
# libs/models/run.py (extrait)
class Run(Base):
    __tablename__ = "run"
    id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid4)
    project_id: Mapped[UUID] = mapped_column(ForeignKey("project.id"))
    asset_id: Mapped[UUID] = mapped_column(ForeignKey("test_asset.id"))
    target_id: Mapped[UUID] = mapped_column(ForeignKey("target.id"))
    state: Mapped[RunState] = mapped_column(Enum(RunState), default=RunState.QUEUED)
    result: Mapped[Result | None]        # CONFORME, NON_CONFORME, INFORMATIF, NON_FIABLE
```

```
    validation: Mapped[Validation]       # EN_ATTENTE, VALIDE, REFUSE
    idempotency_key: Mapped[str]
    __table_args__ = (UniqueConstraint("project_id", "idempotency_key"),)
```

```
# migrations : commandes
alembic revision --autogenerate -m "run et run_event"
alembic upgrade head      # appliquer
alembic downgrade base    # vérifier la réversibilité (obligatoire en CI)
```

Pièges fréquents

- Modifier une migration déjà fusionnée : créer une nouvelle migration à la place.
- Stocker un secret dans run_param : on ne stocke qu'une référence au coffre.
- Oublier de dater en UTC : tous les horodatages sont en UTC, la conversion se fait dans l'interface.

Livrables : migrations, modèles, script de données de démonstration, diagramme entité-relation dans docs/.

Critère de fin : « alembic upgrade head » puis « downgrade base » passent sur une base vide ; les contraintes sont couvertes par des tests.

TABLE	RÔLE	COLONNES CLÉS
project	Projet ou application testée	id, nom, équipe, quotas
environment	Environnement d'un projet	id, project_id, type, fenêtres autorisées
target	Cible autorisée à recevoir des tests	id, environment_id, hôte, type de test autorisé, charge max
test_asset	Script versionné (plan JMeter, suite Selenium)	id, project_id, type, version, empreinte, chemin objet
run	Une exécution	id, asset_id, target_id, état, résultat, validation, clé d'idempotence, demandeur
run_param	Paramètres de l'exécution	run_id, nom, valeur ou référence de secret
run_event	Journal des changements d'état	run_id, horodatage, ancien état, nouvel état, cause
artifact	Fichier produit	run_id, type, chemin objet, empreinte
threshold	Seuil de quality gate	id, project_id, environnement, métrique, opérateur, valeur, bloquant
schedule	Planification	id, asset_id, expression, fuseau, actif
webhook_sub	Déclencheur ou abonné	id, projet, secret (référence), événements
audit_log	Traçabilité	horodatage, acteur, action, objet, adresse

ÉTAPE 2

API REST, authentification, rôles

Objectif : Exposer toutes les capacités par une API documentée, protégée et testable, avant de construire l'interface.

À faire (vue d'ensemble)

- Valider le jeton OIDC et en extraire l'identité et les groupes ; fournir une dépendance FastAPI « require_role ».
- Implémenter les routes de la liste ci-dessous, avec pagination par curseur et réponses d'erreur normalisées (code, message, identifiant de corrélation).
- Appliquer les droits par projet côté serveur sur chaque route ; ne jamais s'appuyer sur un filtrage de l'interface.
- Écrire chaque action sensible dans audit_log, dans la même transaction que l'action.
- Publier le schéma OpenAPI et générer le client TypeScript en pipeline.

Pourquoi : l'API est le seul point d'entrée. Si elle est correcte et testée, l'interface et les pipelines du client ne peuvent pas contourner les règles.

Pas à pas

1. Écrire la dépendance d'authentification : validation de la signature du jeton OIDC (clés publiques de Keycloak, mises en cache), de l'émetteur, de l'audience et de l'expiration.
2. Extraire les groupes du jeton et les traduire en rôles de la plateforme ; un utilisateur sans rôle reçoit 403.
3. Écrire la couche de droits par projet : toute requête de lecture ou d'écriture est filtrée par les projets de l'utilisateur, dans la requête SQL elle-même.
4. Définir les schémas Pydantic d'entrée et de sortie pour chaque route (aucun objet de base n'est renvoyé tel quel).
5. Normaliser les erreurs : un seul format, avec un identifiant de corrélation repris dans les journaux.
6. Écrire l'écriture d'audit comme un service transactionnel appelé dans chaque route de modification.
7. Écrire les tests de droits : une matrice rôle × route générée automatiquement, qui échoue si une route n'a pas de test.

```
# format d'erreur unique
{ "error": { "code": "TARGET_NOT_ALLOWED",
             "message": "La cible n'est pas autorisée pour ce type de test.",
             "correlation_id": "7f3c…", "details": {"target": "recette-appli-x"} } }
```

```
# dépendance de rôle (extrait)
def require_role(*roles):
    async def dep(user=Depends(current_user)):
        if not set(roles) & user.roles:
            await audit.denied(user, request.url.path)   # le refus est tracé
            raise HTTPException(403, error("FORBIDDEN"))
        return user
    return dep
```

RÔLE	PEUT	NE PEUT PAS
Administrateur	Tout, y compris l'arrêt d'urgence et les rôles	Valider un résultat à la place d'un tech lead
Responsable	Gérer équipes, projets, quotas	Lancer des tests de charge sans autorisation
Tech lead	Valider, définir les seuils	Modifier les rôles
Développeur	Consulter, lancer sur les cibles autorisées	Modifier les seuils, valider
Testeur	Déposer des scripts, lancer, annuler	Administrer
Exploitation	Arrêt d'urgence, supervision, agents	Modifier les scripts

RÔLE	PEUT	NE PEUT PAS
Auditeur	Lire les preuves, exporter	Toute écriture

Pièges fréquents

- Faire confiance à l'identifiant de projet envoyé par le client sans le recouper avec les droits de l'utilisateur.
- Renvoyer des messages d'erreur contenant des détails internes (requête SQL, chemin de fichier).
- Oublier le refus tracé : un refus répété est un signal de sécurité.

Livrables : API déployée en local, schéma OpenAPI, tests de droits (un test par rôle et par route sensible).

Critère de fin : un lecteur ne peut ni lancer ni modifier ; un développeur ne lance que sur les cibles autorisées ; les refus sont tracés.

ROUTE	RÔLE MINIMAL	USAGE
POST /projects, GET /projects	Responsable / lecture	Gérer les projets
POST /assets	Testeur	Déposer un script ; calcul de l'empreinte et versionnage
POST /runs	Développeur autorisé	Demander un lancement (en-tête Idempotency-Key obligatoire)
POST /runs/{id}/cancel	Testeur	Annuler ; déclenche l'arrêt côté agent
POST /runs/{id}/validate	Tech lead	Validation humaine du résultat
GET /runs/{id}, /events, /artifacts	Lecture	Suivi et résultats
GET /runs/{id}/gate	Lecture / pipeline	Verdict du quality gate
POST /hooks/{id}	Signature HMAC	Déclenchement par webhook signé
PUT /thresholds	Tech lead	Définir les seuils bloquants ou informatifs
POST /admin/kill	Administrateur / exploitation	Arrêt d'urgence global ou par cible

```
@router.post("/runs", status_code=202)
async def create_run(body: RunIn, idem: str = Header(alias="Idempotency-Key"),
                     user=Depends(require_role("developer")), db=Depends(session)):
    existing = await runs.by_idempotency(db, body.project_id, idem)
    if existing:                          # même appel répété : même réponse
        return RunOut.from_orm(existing)
    await policy.check(db, user, body)  # droits, cible autorisée, quota, plage horaire
    run = await runs.create(db, body, idem, requested_by=user.id, state="QUEUED")
    await audit.write(db, user, "run.create", run.id)
    await queue.enqueue(run.id)
    return RunOut.from_orm(run)
```

ÉTAPE 3

Orchestrateur : états, file, verrous, idempotence

Objectif : Rendre l'exécution fiable : un seul propriétaire de l'état, des verrous corrects, aucune double exécution.

À faire (vue d'ensemble)

- Implémenter la machine à états comme une table de transitions autorisées ; toute transition hors table lève une erreur.
- Prendre un verrou consultatif PostgreSQL par cible avant d'envoyer un test de charge ; relâcher le verrou sur tout état terminal, y compris l'expiration.
- Mettre en place les délais : attente en file, démarrage de l'agent, durée maximale d'exécution, silence de l'agent (battement de cœur).
- Sur perte de l'agent ou redémarrage de l'orchestrateur : passer l'exécution en INDETERMINATE et alerter, sans la relancer.
- Implémenter le kill switch : drapeau global lu avant chaque envoi, plus un ordre d'arrêt diffusé aux agents.
- Compter les quotas (exécutions simultanées par projet, par agent, global) ; au-delà, l'exécution reste en file.

Pourquoi : c'est le cœur de la fiabilité. Les cinq règles de conception du chapitre A s'appliquent ici ; chaque défaut dans cette étape se paie en résultats faux.

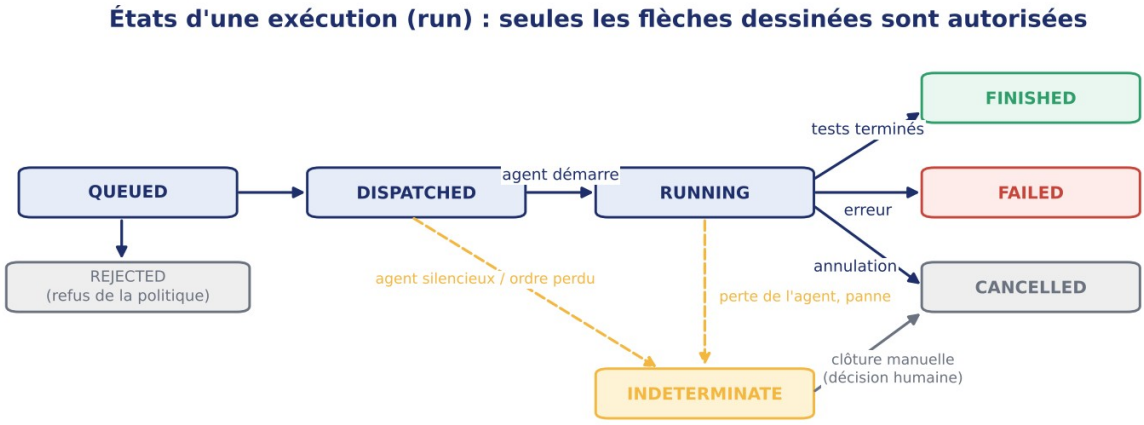


Figure 2. Machine à états d'une exécution.

Contrôles avant tout envoi à un agent : le premier qui échoue arrête la demande

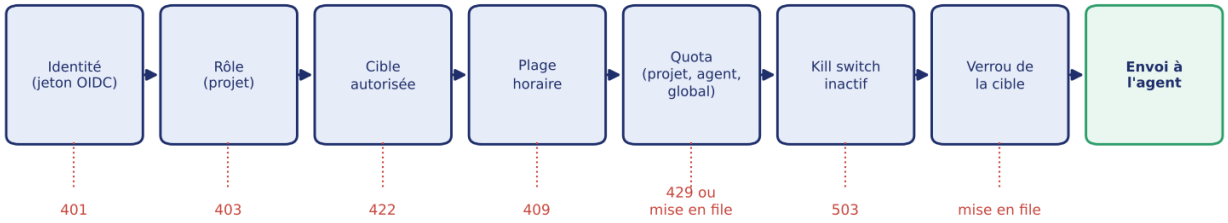


Figure 3. Chaîne de contrôles avant l'envoi à un agent, avec le code de retour de chaque refus.

Pas à pas

1. Implémenter la table de transitions et la fonction move() ; aucune autre fonction n'écrit la colonne state.
2. Implémenter la chaîne de contrôles dans l'ordre de la figure 3 ; chaque contrôle est une classe avec un test dédié.
3. Écrire la prise de verrou par cible avec un verrou consultatif PostgreSQL ; le verrou est pris dans la transaction qui passe l'exécution à DISPATCHED.
4. Écrire le surveillant (watchdog) : une tâche périodique qui repère les exécutions RUNNING dont l'agent n'a pas émis de battement de cœur depuis plus de trois intervalles, et les passe en INDETERMINATE.
5. Écrire le kill switch : un drapeau en base lu à chaque envoi, et un message d'arrêt diffusé aux agents concernés.
6. Écrire le calcul de quotas avec des compteurs lus dans la même transaction que la décision.
7. Écrire les tests de panne et les faire tourner en boucle (cent fois) pour déceler les courses entre concurrents.

```
# verrou de cible (extrait)
async def try_lock_target(db, target_id) -> bool:
    key = int.from_bytes(target_id.bytes[:8], "big", signed=True)
    row = await db.execute(text("select pg_try_advisory_xact_lock(:k)"), {"k": key})
    return row.scalar()           # False : la cible est occupée, l'exécution reste QUEUED
```

```
# surveillant d'agents silencieux (extrait)
async def watchdog(db):
    stale = await runs.running_without_heartbeat(db, max_age_s=3 * HEARTBEAT_S)
    for run in stale:
        move(run, "INDETERMINATE", cause="agent_silent")   # jamais de relance automatique
        await alerts.send("run_indeterminate", run.id)
```

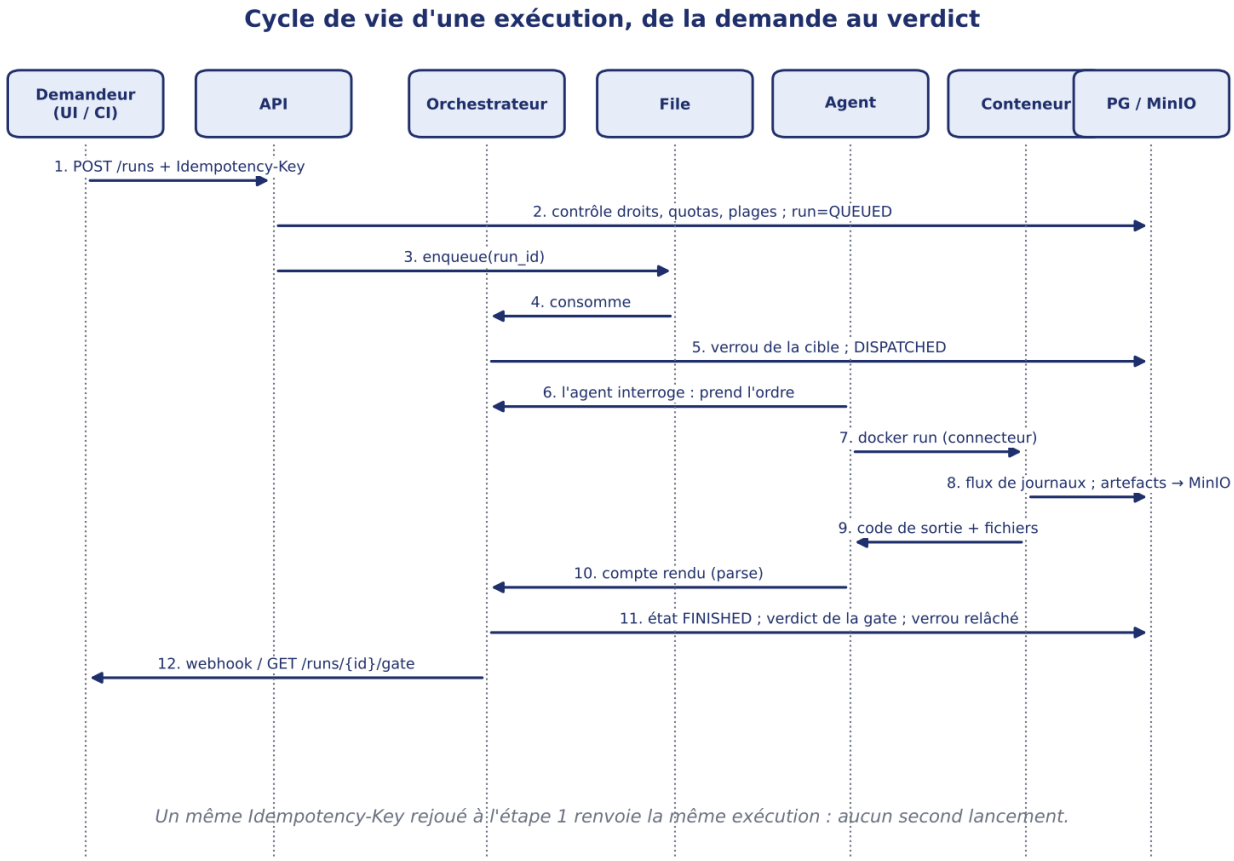


Figure 4. Cycle de vie complet d'une exécution entre les composants.

Pièges fréquents

- Relancer automatiquement une exécution perdue : un test de charge relancé peut doubler la charge sur une application sensible.
- Prendre le verrou hors de la transaction qui change l'état : deux orchestrateurs peuvent alors dispatcher sur la même cible.
- Libérer le verrou seulement sur FINISHED : il faut le libérer aussi sur FAILED, CANCELLED, INDETERMINATE et à l'expiration.

Livrables : service orchestrateur, table de transitions, tests de simulation de pannes.

Critère de fin : tests de panne : agent coupé en cours de test, redémarrage de l'orchestrateur, double appel API, deux lancements concurrents sur la même cible. Aucun ne produit de double exécution ni de relancement automatique.

```
TRANSITIONS = {
  "QUEUED":        {"DISPATCHED", "CANCELLED", "REJECTED"},
  "DISPATCHED":    {"RUNNING", "CANCELLED", "INDETERMINATE"},
  "RUNNING":       {"FINISHED", "FAILED", "CANCELLED", "INDETERMINATE"},
```

```
  "FINISHED":      set(), "FAILED": set(), "CANCELLED": set(),
  "INDETERMINATE": {"CANCELLED"},        # clôture manuelle uniquement
}
def move(run, new, cause):
    if new not in TRANSITIONS[run.state]:
        raise IllegalTransition(run.state, new)
    events.append(run.id, run.state, new, cause); run.state = new
```

ÉTAPE 4

Agents d'exécution et contrat de connecteur

Objectif : Isoler l'exécution : l'agent reçoit un ordre, lance un conteneur et rend compte ; les connecteurs sont interchangeables.

À faire (vue d'ensemble)

- Écrire le démon d'agent : enregistrement auprès de l'orchestrateur avec un jeton de courte durée, battement de cœur, récupération des ordres.
- Définir le SDK de connecteur (interface ci-dessous) et le mécanisme de chargement par type de test.
- Lancer chaque exécution dans un conteneur éphémère : utilisateur non privilégié, système de fichiers en lecture seule sauf le dossier de travail, réseau limité aux cibles autorisées, limites de mémoire et de processeur.
- Remonter les journaux en flux avec masquage des secrets, puis téléverser les artefacts vers MinIO avec leur empreinte.
- Prévoir l'arrêt propre : signal d'arrêt, délai de grâce, arrêt forcé, état final remonté même en cas d'arrêt.

Pourquoi : l'isolation protège la plateforme et les applications du client contre un script défectueux ou malveillant.

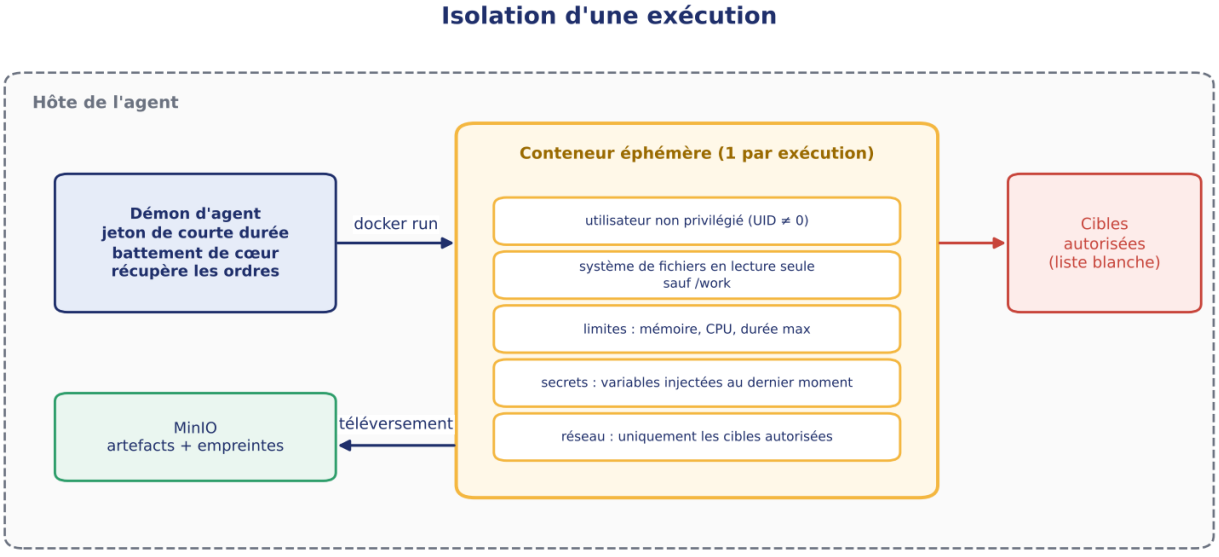


Figure 5. Isolation d'une exécution : ce que le conteneur peut et ne peut pas faire.

Pas à pas

1. Écrire l'enregistrement de l'agent : échange d'un jeton d'enrôlement à usage unique contre un jeton de courte durée, renouvelé automatiquement.

2. Écrire la boucle principale : toutes les deux secondes, demander un ordre à l'orchestrateur ; émettre un battement de cœur toutes les dix secondes.

3. Écrire le chargeur de connecteurs : un connecteur par type ; le type inconnu est refusé.

4. Lancer le conteneur avec les options d'isolation ci-dessous ; l'agent ne monte jamais le socket Docker dans le conteneur d'exécution.

5. Diffuser les journaux ligne à ligne vers l'API en masquant les valeurs de secrets connues.

6. Téléverser les artefacts vers MinIO avec calcul d'empreinte SHA-256, puis déclarer l'artefact.

7. Gérer l'arrêt : SIGTERM, délai de grâce, SIGKILL ; remonter toujours un état final, y compris quand le conteneur a été tué.

```
docker run --rm --name run-$RUN_ID \
  --user 10001:10001 --read-only --tmpfs /tmp \
  -v $WORKDIR:/work:rw --workdir /work \
  --memory 4g --cpus 2 --pids-limit 512 \
  --cap-drop ALL --security-opt no-new-privileges \
  --network run-net-$TARGET_ID \
  -e TARGET_URL -e TOKEN_REF \
  jmeter-runner:5.6.3-2026.10 jmeter -n -t plan.jmx -l results.jtl
```

```
# boucle d'agent (extrait)
while True:
    order = await orch.poll(agent_id)
    if order: asyncio.create_task(execute(order))
    await orch.heartbeat(agent_id, running=list(active))
    await asyncio.sleep(2)
```

Pièges fréquents

- Donner accès au socket Docker au conteneur de test : cela équivaut à donner les droits administrateur de l'hôte.
- Laisser le réseau ouvert : la liste blanche de cibles est une règle de sécurité, pas une option.
- Oublier le téléversement quand le test échoue : les artefacts d'un échec sont les plus utiles.

Livrables : agent, SDK connecteur, image de base, connecteur « écho » de démonstration.

Critère de fin : le connecteur écho passe de bout en bout : lancement depuis l'API, exécution en conteneur, artefact visible, arrêt à la demande.

```
class Connector(Protocol):
    kind: str                                  # "jmeter", "selenium"
    def validate(self, spec: RunSpec) -> list[Problem]: ...   # avant tout lancement
    def prepare(self, spec: RunSpec, workdir: Path) -> Command: ...
    def parse(self, workdir: Path) -> Result: ...             # métriques et statuts normalisés
    def artifacts(self, workdir: Path) -> list[Path]: ...
    def stop(self, handle: Handle, grace_s: int) -> None: ...
```

ÉTAPE 5

Connecteur JMeter

Objectif : Exécuter des plans JMeter en mode non graphique, collecter les résultats et détecter la saturation du générateur.

À faire (vue d'ensemble)

- Valider le plan à l'import : version JMeter compatible, extensions et plugins présents dans l'image, aucun chemin absolu, aucun secret en clair.
- Construire la ligne de commande : « jmeter -n -t plan.jmx -l results.jtl -e -o report/ » avec les paramètres passés par -J, jamais en dur dans le plan.
- Lire le JTL en flux : débit, temps de réponse (moyenne, p90, p95, p99), taux d'erreur par échantillon et par code.
- Surveiller le générateur : processeur, mémoire, temps de ramassage de mémoire, écart entre débit demandé et obtenu. En cas de saturation, marquer l'exécution « résultat non fiable » même si le test s'est terminé.
- Calculer le verdict contre les seuils de la table threshold ; stocker le détail de chaque seuil évalué.
- Gérer l'exécution répartie (plusieurs générateurs) : démarrage synchronisé, agrégation des JTL, seuil de non-fiabilité propre.

Pourquoi : un test de charge dont le générateur est saturé produit des chiffres qui accusent l'application à tort. La détection de saturation est la fonction la plus importante du connecteur.

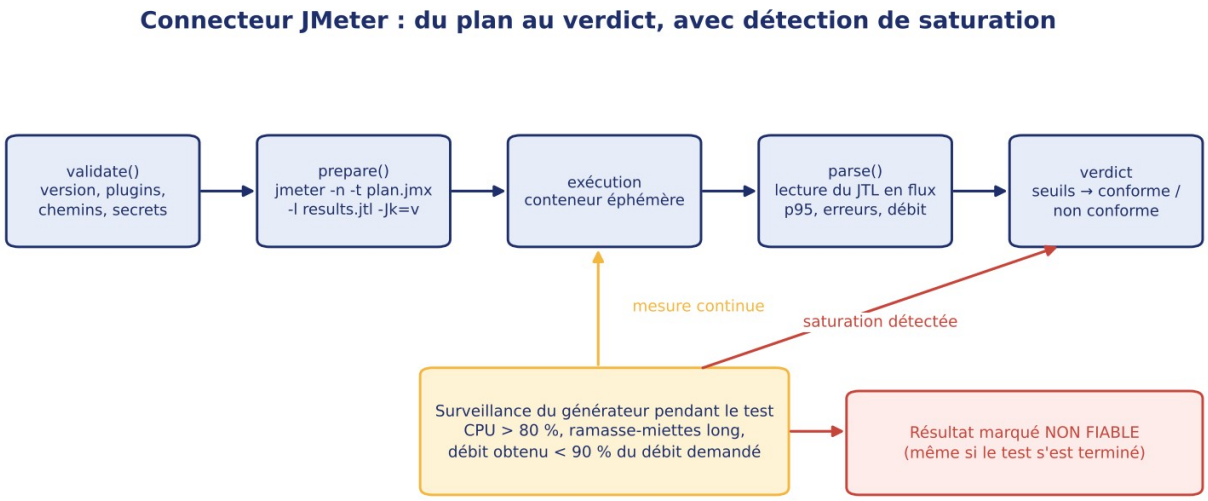


Figure 6. Connecteur JMeter, du plan au verdict.

Pas à pas

1. Construire l'image : base Java, JMeter à version précise, plugins nécessaires, utilisateur non privilégié, aucun accès Internet à l'exécution.
2. Écrire validate() : lecture du fichier .jmx comme XML, détection des chemins absolus, des identifiants en clair, des plugins absents et du nombre de threads supérieur au maximum de la cible.

3. Écrire prepare() comme dans l'exemple de l'étape précédente ; tout paramètre arrive par -J, jamais par modification du plan.
4. Écrire le lecteur de JTL en flux (CSV) : agrégation par fenêtre d'une seconde et calcul des centiles avec une structure à mémoire bornée.
5. Écrire le moniteur de générateur : lecture de la consommation CPU et mémoire du conteneur, temps de ramassage de mémoire JVM, débit obtenu comparé au débit visé.
6. Écrire le calcul de verdict contre les seuils, et le marquage NON_FIABLE si le moniteur a signalé une saturation.
7. Valider avec trois plans de référence : léger, nominal et volontairement trop lourd pour la machine.

```
# détection de saturation (extrait)
def generator_saturated(s: GeneratorStats) -> bool:
    return (s.cpu_pct_p95 > 80
            or s.gc_pause_ms_p95 > 500
            or s.achieved_rps < 0.90 * s.target_rps)   # seuils paramétrables par projet
```

```
# lecture du JTL en flux (extrait)
def read_jtl(path):
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            yield Sample(ts=int(row["timeStamp"]), ms=int(row["elapsed"]),
                         ok=row["success"] == "true", code=row["responseCode"], label=row["label"])
```

MÉTRIQUE	CALCUL	REMARQUE
Débit	échantillons par seconde sur la fenêtre stable	Exclure la montée et la descente en charge
p95 / p99	centile des temps de réponse	Jamais la moyenne seule
Taux d'erreur	échantillons en échec / total	Par libellé et par code de réponse
Saturation	voir le code ci-dessus	Marque le résultat NON_FIABLE

Pièges fréquents

- Lancer l'interface graphique de JMeter pour les tests réels : seul le mode non graphique est autorisé.
- Calculer les centiles sur toute la durée, montée en charge comprise : les résultats sont faussés.
- Charger tout le JTL en mémoire : un test d'une heure produit des gigaoctets.

Livrables : connecteur JMeter, image Docker versionnée, jeu de plans de référence sur la cible factice.

Critère de fin : un plan de référence produit des résultats identiques (à la tolérance près) sur trois exécutions ; un plan volontairement trop lourd est détecté comme générateur saturé.

```
def prepare(self, spec, workdir):
    args = ["jmeter", "-n", "-t", str(spec.asset_path), "-l", str(workdir/"results.jtl"),
            "-e", "-o", str(workdir/"report")]
    for k, v in spec.public_params.items():        # paramètres non secrets
        args += [f"-J{k}={v}"]
    return Command(args, env=spec.secret_env_refs, workdir=workdir)
```

ÉTAPE 6

Connecteur Selenium

Objectif : Exécuter des suites de parcours fonctionnels, produire des résultats par test et des preuves (captures, journaux).

À faire (vue d'ensemble)

- Choisir le moteur de suites (pytest-selenium ou JUnit selon les scripts du client) ; valider à l'import le format du rapport JUnit.
- Utiliser Selenium Grid ou des navigateurs en conteneur ; version de navigateur et de pilote figées dans l'image.
- Capturer écran, journal de console et trace réseau sur échec ; nommer les artefacts par test.
- Normaliser le rapport JUnit en statuts : réussi, en échec, non exécuté, instable.
- Autoriser une nouvelle tentative uniquement pour Selenium, et marquer « instable » un test qui passe à la seconde tentative.
- Refuser toute option de montée en charge ; rejeter un lancement qui demande plusieurs utilisateurs virtuels.

Pourquoi : Selenium vérifie des parcours fonctionnels ; il ne mesure pas la performance sous charge. Le connecteur doit empêcher cet usage.

Pas à pas

1. Fixer le moteur de suites et le format du rapport JUnit avec le client (étape de cadrage).
2. Construire l'image avec navigateur et pilote à version précise ; vérifier à chaque construction que les deux sont compatibles.
3. Écrire validate() : refuser tout paramètre de type utilisateurs virtuels, durée de charge ou débit.
4. Écrire la collecte des preuves : capture d'écran, journal de console, trace réseau, nommées par test et par tentative.
5. Normaliser le rapport JUnit en quatre statuts : réussi, en échec, non exécuté, instable.
6. Implémenter la nouvelle tentative : une seule, uniquement en cas d'échec ; un test réussi à la seconde tentative est marqué instable.
7. Valider avec une suite de référence contenant un test volontairement instable et un test volontairement en échec.

```
# normalisation JUnit (extrait)
def status(case, attempts):
    if case.skipped:               return "NON_EXECUTE"
    if case.failed:                return "ECHEC"      # échec après la nouvelle tentative
    return "INSTABLE" if attempts > 1 else "REUSSI"
```

```
pytest suites/parcours -q --junitxml=work/report.xml \
  --reruns 1 --reruns-delay 2 --screenshot=only-on-failure
```

Pièges fréquents

- Interpréter une lenteur de navigateur comme un résultat de performance : Selenium ne mesure pas la performance de l'application sous charge.
- Laisser tourner des navigateurs sans limite de mémoire : ils saturent l'hôte de l'agent.
- Masquer les tests instables : ils doivent apparaître dans le rapport, avec leur taux.

Livrables : connecteur Selenium, image avec navigateurs, suite de référence sur la cible factice.

Critère de fin : les comptes du rapport plateforme égalent ceux du JUnit source ; un test volontairement instable est marqué instable.

ÉTAPE 7

Workflows, planification, webhooks, quality gates

Objectif : Enchaîner les tests, les déclencher automatiquement et exposer un verdict que les pipelines du client peuvent consommer.

À faire (vue d'ensemble)

- Définir le format de workflow en YAML (étapes, dépendances, condition de poursuite) et un validateur de schéma.
- Implémenter le planificateur (expression cron, fuseau, interdiction de chevauchement sur une même cible).
- Implémenter les webhooks entrants : vérification HMAC-SHA256 avec comparaison à temps constant, horodatage maximal de cinq minutes contre le rejeu, clé d'idempotence de l'événement.
- Implémenter les webhooks sortants : signature, nouvelles tentatives avec délai croissant, journal des livraisons.
- Évaluer le quality gate à la fin de chaque exécution : seuil bloquant non respecté donne « non conforme », seuil informatif donne « informatif ». Exposer le verdict par GET /runs/{id}/gate et par webhook.
- Fournir au client un exemple d'appel depuis GitHub Actions, GitLab CI, Jenkins et Azure DevOps (script d'appel et d'attente du verdict).

Pourquoi : c'est ce qui transforme la plateforme en maillon de la chaîne de livraison : un pipeline la déclenche et lit son verdict.

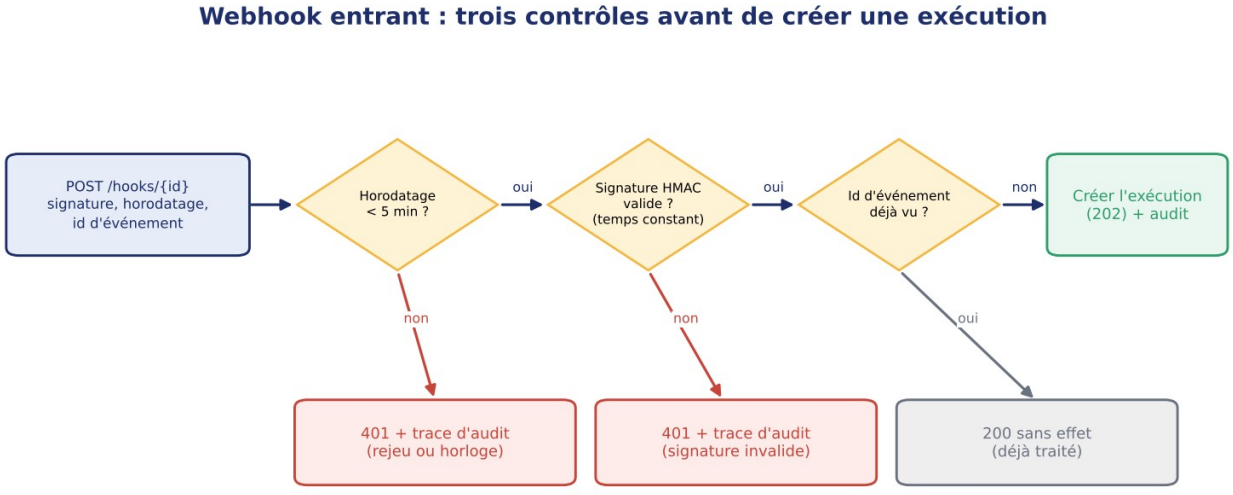


Figure 7. Contrôles du webhook entrant.

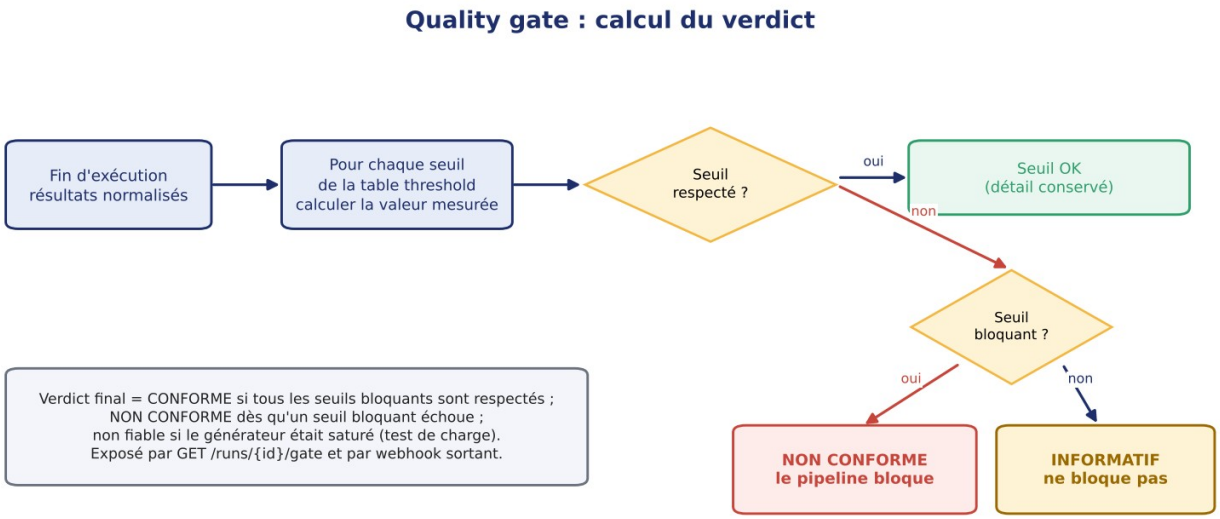


Figure 8. Calcul du verdict de quality gate.

Pas à pas

1. Écrire le schéma JSON du workflow et le validateur ; un workflow invalide est refusé avec la ligne en cause.
2. Écrire l'exécuteur de workflow : un graphe d'étapes avec dépendances et conditions, chaque étape étant une exécution ordinaire.
3. Écrire le planificateur : expression cron, fuseau, interdiction de chevauchement sur une même cible, saut tracé si la précédente n'est pas terminée.
4. Écrire le webhook entrant avec les contrôles de la figure 7 et un secret par abonnement, stocké dans le coffre.
5. Écrire le webhook sortant : signature HMAC, nouvelles tentatives à délai croissant (1, 5, 30 minutes), journal de livraison consultable.
6. Écrire l'évaluateur de quality gate (figure 8) et l'endpoint GET /runs/{id}/gate avec attente optionnelle (long polling).
7. Fournir des scripts d'exemple d'appel pour chaque outil CI/CD du client.

```
# vérification HMAC (extrait)
def verify(secret: bytes, body: bytes, ts: str, sig: str) -> bool:
    if abs(time.time() - int(ts)) > 300:                    # rejeu
        return False
    expected = hmac.new(secret, ts.encode() + b"." + body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, sig)               # temps constant
```

```
# GitHub Actions : lancer et attendre le verdict (exemple fourni au client)
- name: Tests de performance
  run: |
    RUN=$(curl -sS -X POST $ORCHESTRA/runs -H "Authorization: Bearer $TOKEN" \
          -H "Idempotency-Key: $GITHUB_RUN_ID-perf" -d @perf.json | jq -r .id)
    curl -sS "$ORCHESTRA/runs/$RUN/gate?wait=900" -H "Authorization: Bearer $TOKEN" \
      | jq -e '.verdict == "CONFORME"'      # code de sortie non nul : le pipeline s'arrête
```

Pièges fréquents

- Comparer la signature avec == : utiliser une comparaison à temps constant.
- Accepter un webhook sans horodatage : le rejeu devient possible.
- Bloquer un pipeline sur un test informatif : seul un seuil déclaré bloquant doit produire NON CONFORME.

Livrables : moteur de workflow, planificateur, webhooks, exemples d'intégration.

Critère de fin : dix appels dont cinq mal signés : cinq refusés et tracés, cinq exécutions ; un seuil dépassé produit « non conforme » côté API et côté webhook.

```
name: recette-appli-x
steps:
  - id: perf
    kind: jmeter
    asset: plan-charge-v3
    target: recette-appli-x
    params: {threads: 200, duration: 600}
  - id: parcours
    kind: selenium
    asset: suite-parcours-v5
    target: recette-appli-x
    needs: [perf]
    when: perf.result != "non_conforme"
gate:
  blocking: [perf.p95_ms <= 800, perf.error_rate <= 0.01]
  informative: [parcours.flaky_count <= 2]
```

ÉTAPE 8

Interface web

Objectif : Offrir aux testeurs un parcours simple : choisir, lancer, suivre, valider. L'interface ne contient aucune règle métier.

À faire (vue d'ensemble)

- Écrans : projets et environnements, scripts et versions, lancement (formulaire généré à partir des paramètres du script), suivi en direct, résultats avec tendance, validation, administration des seuils et des rôles.
- Suivi en direct par SSE ou WebSocket depuis run_event ; courbes de débit et de temps de réponse alimentées par l'API.
- Afficher séparément état, résultat et validation ; afficher clairement « résultat non fiable » et « indéterminé ».
- Masquer les actions interdites selon le rôle (confort seulement, le contrôle reste côté serveur).
- Accessibilité de base, adaptation à l'écran de portable, textes en français.

Pourquoi : l'interface fait gagner du temps aux testeurs ; elle ne porte aucune règle, elle montre l'état que l'orchestrateur a décidé.

Pas à pas

1. Générer le client TypeScript depuis le schéma OpenAPI ; ne jamais écrire à la main un appel d'API.
2. Mettre en place le routage, la session OIDC et le jeton d'accès en mémoire (pas dans le stockage du navigateur).
3. Construire les écrans dans l'ordre du tableau ci-dessous ; chacun a son test de bout en bout.

4. Brancher le suivi en direct sur le flux d'événements d'une exécution, avec reconnexion automatique.
5. Afficher les trois dimensions de verdict séparément, avec les mêmes libellés que le chapitre 4 de l'offre.
6. Contrôler l'accessibilité clavier et les contrastes ; vérifier l'affichage sur écran de portable.

ÉCRAN	RÔLE VISÉ	TEST DE BOUT EN BOUT
Projets et environnements	Responsable	Créer un projet avec deux environnements
Scripts et versions	Testeur	Déposer un plan, voir l'empreinte et la version
Lancement	Développeur, testeur	Lancer sur la cible factice ; refus sur une cible non autorisée
Suivi en direct	Tous	Voir les états changer et les courbes avancer
Résultats et tendances	Tous	Comparer deux campagnes
Validation	Tech lead	Valider un résultat ; le verdict reste distinct
Administration	Administrateur	Modifier un rôle, un seuil, un quota ; trace d'audit présente

ÉTAT TECHNIQUE	LIBELLÉ AFFICHÉ	COULEUR
QUEUED	En attente	Gris
RUNNING	En cours	Bleu
FINISHED + CONFORME	Conforme	Vert
FINISHED + NON_CONFORME	Non conforme	Rouge
FINISHED + NON_FIABLE	Résultat non fiable	Orange
INDETERMINATE	Indéterminé : décision humaine requise	Orange

Pièges fréquents

- Recalculer un verdict dans l'interface : il vient toujours de l'API.
- Stocker le jeton d'accès dans le stockage local du navigateur.
- Afficher « terminé » pour un test dont le générateur était saturé.

Livrables : interface déployée, tests de composants, parcours de bout en bout automatisés.

Critère de fin : un testeur sans aide lance une exécution et lit son verdict en moins de cinq minutes lors de la recette d'utilisabilité.

ÉTAPE 9

Sécurité, supervision, sauvegarde

Objectif : Rendre la plateforme exploitable et défendable avant tout usage sur un environnement réel.

À faire (vue d'ensemble)

- Secrets : lecture depuis le coffre au dernier moment, injection par variable d'environnement dans le conteneur, masquage dans les journaux (test unitaire avec secrets factices).
- Authentification multifacteur pour les administrateurs, sessions courtes, révocation des jetons d'agent.
- Cibles autorisées : liste blanche stricte, contrôle de l'adresse résolue pour éviter la redirection vers une cible non autorisée.

- Images : analyse des vulnérabilités en pipeline, versions figées, liste des dépendances produite à chaque construction.
- Métriques Prometheus : exécutions par état, durée en file, saturation des agents, échecs de webhooks. Alertes sur exécutions indéterminées et agents silencieux.
- Sauvegarde chiffrée de PostgreSQL et de MinIO, procédure de restauration écrite et testée en recette.

Pourquoi : avant le premier usage sur un environnement réel, la plateforme doit prouver qu'elle ne fuit pas de secret et qu'elle se restaure.

Pas à pas

1. Brancher le coffre de secrets ; écrire le test de fuite : injecter des secrets factices et vérifier leur absence des journaux, rapports, artefacts et base.
2. Activer l'authentification multifacteur pour le groupe administrateur dans Keycloak.
3. Écrire la liste blanche de cibles avec contrôle de l'adresse résolue au moment de l'exécution.
4. Activer l'analyse des vulnérabilités des images et produire la liste des composants à chaque construction.
5. Exposer les métriques du tableau ci-dessous et créer les tableaux de bord et les alertes.
6. Écrire et tester la sauvegarde chiffrée quotidienne et la restauration sur un environnement vierge.
7. Rédiger le plan de réponse aux incidents : qui arrête quoi, qui prévient le client.

MÉTRIQUE	TYPE	ALERTE
orchestra_runs_total{state}	compteur	Pic de FAILED ou INDETERMINATE
orchestra_queue_wait_seconds	histogramme	Attente supérieure au délai convenu
orchestra_agent_heartbeat_age_seconds	jauge	Supérieure à trois intervalles
orchestra_generator_saturation_total	compteur	Information : résultats non fiables
orchestra_webhook_failures_total	compteur	Échecs répétés de livraison
orchestra_killswitch_active	jauge	Toujours alerter

```
# sauvegarde et restauration (exemple)
pg_dump -Fc orchestra | age -r $CLE_PUBLIQUE > pg-$(date +%F).dump.age
mc mirror --overwrite minio/orchestra-artefacts /sauvegarde/artefacts
# restauration
age -d -i $CLE_PRIVEE pg-2026-10-23.dump.age | pg_restore -d orchestra_restore --clean
```

Pièges fréquents

- Tester la sauvegarde sans tester la restauration : seule une restauration réussie prouve quelque chose.
- Laisser des jetons d'agent valides après le retrait d'un agent : les révoquer.
- Journaliser les en-têtes des requêtes : ils contiennent le jeton d'accès.

Livrables : configuration de sécurité, tableaux Grafana, alertes, procédure de restauration.

Critère de fin : restauration complète sur un environnement vierge réussie ; aucun secret retrouvé dans les journaux, les rapports ni la base lors du test de fuite.

ÉTAPE 10

Tests de la plateforme et recette

Objectif : Prouver que les règles de fiabilité tiennent, avant la recette client.

À faire (vue d'ensemble)

- Pyramide : tests unitaires sur la machine à états et les règles de quota ; tests d'intégration avec PostgreSQL, Redis et MinIO réels en conteneur ; tests de bout en bout sur la cible factice.
- Tests de panne automatisés : agent tué en cours d'exécution, base indisponible quelques secondes, file vidée, horloge décalée.
- Tests de sécurité : droits par rôle et par projet, webhooks mal signés ou rejoués, tentative de cible non autorisée.
- Dérouler les critères de recette R1 à R21 de l'offre technique et conserver les preuves.
- Rédiger la documentation d'exploitation, d'administration et la spécification des connecteurs.

Pourquoi : les critères de recette sont contractuels ; les tests automatisés rendent leur démonstration répétable.

FAMILLE DE TESTS	OUTIL	CE QU'ELLE PROUVE	CRITÈRES DE RECETTE LIÉS
Unitaires	pytest	Transitions, quotas, calcul de verdict, détection de saturation	R16, R18, R19
Intégration	pytest + conteneurs	Base, file, MinIO réels ; idempotence ; verrous	R1 à R6
Pannes	pytest + scripts de chaos	Agent tué, base coupée, redémarrage de l'orchestrateur	R7 à R10
Sécurité	pytest + outils d'analyse	Droits, webhooks, cibles, fuite de secrets	R11 à R14, R17
Bout en bout	Playwright ou Selenium sur l'interface	Parcours testeur complet	R15, R20
Restauration	script et procédure	Sauvegarde puis restauration complète	R21

La correspondance exacte entre critères et familles de tests est à vérifier avec le tableau R1 à R21 de l'offre technique avant la recette.

Pas à pas

1. Écrire les tests de panne comme des scénarios déclaratifs (qui casse quoi, à quel moment, résultat attendu).
2. Les faire tourner dans le pipeline de nuit et pas seulement à la demande.
3. Remplir, critère par critère, le tableau de preuves : commande lancée, résultat, capture, date, auteur.
4. Faire relire le tableau par quelqu'un qui n'a pas écrit le code.
5. Préparer la démonstration de recette sur la cible factice puis sur l'application pilote du client.

```
# scénario de panne (extrait)
def test_agent_killed_mid_run(platform):
    run = platform.launch("jmeter", plan="plan-nominal", target="fake")
    platform.wait_state(run, "RUNNING")
    platform.kill_agent(run.agent_id)
    platform.wait_state(run, "INDETERMINATE", timeout=60)
    assert platform.relaunch_count(run) == 0          # aucun relancement automatique
    assert platform.target_lock_released("fake")
```

Pièges fréquents

- Ne tester que les cas nominaux : la valeur de la plateforme est dans les cas de panne.
- Valider la recette sur une machine de développeur : l'environnement de recette est celui qui compte.

Livrables : suite de tests automatisée dans le pipeline, rapport de recette, documentation.

Critère de fin : tous les critères R1 à R21 sont démontrés sur l'environnement de recette, preuves archivées.

B

Règles d'équipe et suites

Définition de « terminé » pour toute tâche

- Code relu par un pair, types et lint sans erreur, tests ajoutés ou mis à jour.
- Migration réversible si elle touche la base ; schéma OpenAPI régénéré si l'API change.
- Journalisation sans donnée sensible ; événement d'audit ajouté pour toute action de modification.
- Documentation mise à jour (ADR si une décision d'architecture est prise).

Ordre de travail conseillé en parallèle

PENDANT	PROFIL	PEUT AVANCER EN PARALLÈLE
Étapes 1 à 2	Développeur A / B	A : modèle et API. B : compose, Keycloak et interface (maquettes).
Étape 3	Architecte + développeur A	Orchestrateur ; B prépare le SDK de connecteur.
Étapes 5 et 6	Développeur B + ingénieur test de performance	JMeter et Selenium simultanément ; plans et suites de référence écrits par l'ingénieur de test.
Étapes 7 et 8	A : workflows. B : interface	Contrat d'API figé avant la fin de l'étape 6.
Étapes 9 et 10	Toute l'équipe + appui infrastructure	Sécurité, supervision, sauvegarde ; recette.

Lot CI/CD optionnel (après la première version)

- Adaptateurs par outil (GitHub Actions, GitLab CI, Jenkins, Azure DevOps) sur le contrat de webhook et de verdict de l'étape 7.
- Promotion entre environnements : machine à états distincte de celle des exécutions, approbation à deux niveaux, historique des promotions.
- Déploiement en recette et retour arrière : retour de l'image applicative automatique ; les migrations de données irréversibles restent une décision humaine.
- Cet ensemble ne démarre qu'après l'audit de découverte et la décision du client.

Risques à surveiller pendant le développement

RISQUE	RÉFLEXE
Résultats de charge faux à cause d'un générateur saturé	Mesure de saturation dès le premier plan de référence
Doubles lancements sur une cible sensible	Test de concurrence automatisé dès l'étape 3
Secrets dans les journaux	Test de fuite dans le pipeline à chaque fusion
Dérive de périmètre	Toute demande hors matrice MVP passe par l'architecte et le backlog
Dépendance à un outil du client non connu	Ne rien coder d'un adaptateur avant l'audit de découverte

Glossaire

TERME	SENS DANS CE PROJET
Run	Une exécution d'un script sur une cible
Cible	Application ou environnement autorisé à recevoir un test
Connecteur	Module qui sait préparer, lancer et lire les résultats d'un outil (JMeter, Selenium)
Agent	Démon qui exécute les ordres dans un conteneur isolé
Quality gate	Règle de seuils qui produit un verdict que le pipeline lit
Idempotence	Rejouer la même demande ne crée pas de seconde exécution
Verrou de cible	Un seul test de charge à la fois sur une même cible
Kill switch	Arrêt d'urgence global ou par cible
INDETERMINATE	État d'une exécution dont l'issue est inconnue : décision humaine, jamais de relance automatique
JTL	Fichier de résultats de JMeter